

Département Mathématique et Informatique

**Filière : « GLSID - Génie du Logiciel et des Systèmes
Informatiques Distribués »**

Compte rendu

Examen Architecture Distribuée et Middleware

Réalisé par :

HATTABI Youness

Encadré par :

Pr. Mohamed YOUSSEFI

Année universitaire : 2025/2026

Table des matières

1. Architecture Technique du Projet	3
Flux de communication	3
2. Diagramme de Classes	3
3. Implémentation - Couche DAO	4
3.1. Entités JPA	4
3.2. Repositories Spring Data	4
3.3. Données de test	4
4. Couche Service — DTOs et Mappers.....	4
4.1. Principales fonctionnalités	4
4.2. DTOs utilisés	5
4.3. Mappers	5
5. Web Services	5
5.1. Endpoints principaux.....	5
6. Sécurité — Spring Security et JWT	6
6.1. Mécanisme d'authentification	6
6.2. Rôles et autorisations.....	6
6.3. Composants Spring Security.....	7

1. Architecture Technique du Projet

L'application est structurée selon une architecture multi-couches classique inspirée du patron MVC, déployée sur un serveur Spring Boot embarqué. L'ensemble des couches communique verticalement, du frontend (non couvert ici) jusqu'à la base de données.

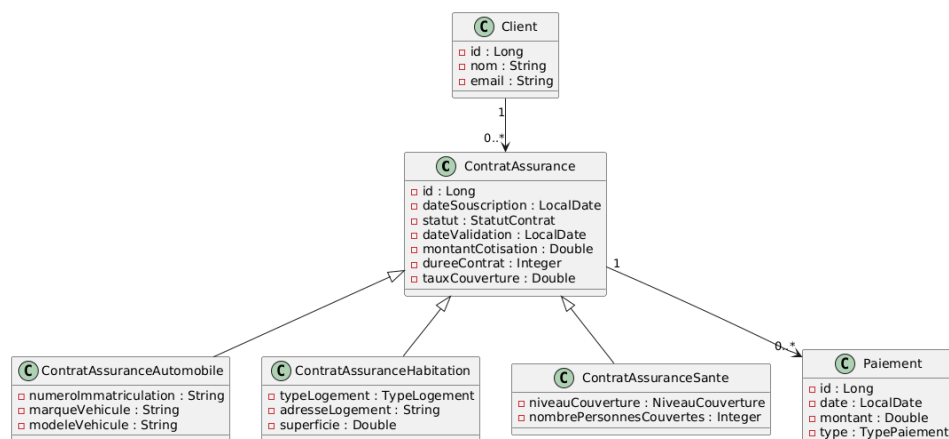
Couche	Technologie	Rôle
Présentation (Web)	Spring MVC / REST Controllers	Exposition des API REST, sérialisation JSON
Sécurité	Spring Security + JWT	Authentification, autorisation par rôles
Service (Métier)	Spring Service + DTOs + Mappers	Logique métier, transformation des données
DAO / Persistance	Spring Data JPA + Hibernate	Accès base de données, requêtes JPQL
Base de données	H2	Stockage des entités

Flux de communication

Le client HTTP envoie des requêtes vers les REST Controllers. Chaque requête est d'abord interceptée par le filtre JWT qui valide le token. Si l'authentification est valide, la requête est transmise à la couche service, qui orchestre la logique métier et délègue l'accès aux données à la couche DAO via les interfaces JPA Repository.

2. Diagramme de Classes

Le diagramme ci-dessous (exprimé en PlantUML) présente les entités principales du domaine et leurs relations. La classe ContratAssurance sert de classe mère aux trois types de contrats via un héritage JPA (SINGLE_TABLE).



3. Implémentation - Couche DAO

La couche DAO repose entièrement sur Spring Data JPA. Les entités sont annotées avec les annotations JPA standard (@Entity, @Inheritance, @ManyToOne, @OneToMany, etc.). Chaque entité dispose de son propre Repository.

3.1. Entités JPA

La classe ContratAssurance est annotée @Inheritance(strategy = InheritanceType.SINGLE_TABLE) afin de stocker chaque type de contrat dans sa propre table tout en mutualisant les colonnes communes. La relation Client → ContratAssurance est de type @OneToMany avec cascade, et ContratAssurance → Paiement également.

3.2. Repositories Spring Data

- ClientRepository extends JpaRepository
- ContratAssuranceAutomobileRepository extends JpaRepository
- ContratAssuranceHabitationRepository extends JpaRepository
- ContratAssuranceSanteRepository extends JpaRepository
- PaiementRepository extends JpaRepository

3.3. Données de test

Un CommandLineRunner injecté dans la classe principale AssuranceApplication peuple la base au démarrage : 3 clients, 2 contrats automobile, 2 contrats habitation, 2 contrats santé et 6 paiements de types variés (mensualité, annuel, exceptionnel).

4. Couche Service — DTOs et Mappers

La couche service est définie par une interface ContratService et son implémentation ContratServiceImpl. Elle s'appuie sur des DTOs (Data Transfer Objects) pour découpler la représentation interne des entités JPA de la représentation exposée via les API REST.

4.1. Principales fonctionnalités

- Gestion des clients : création, mise à jour, suppression, recherche par nom ou email
- Gestion des contrats : souscription, validation, résiliation, recherche par client ou statut
- Gestion des paiements : enregistrement d'un paiement, historique par contrat
- Calcul du montant total des paiements par contrat

4.2. DTOs utilisés

- ClientDTO
- ContratAssuranceDTO
- ContratAssuranceAutomobileDTO
- ContratAssuranceHabitationDTO
- ContratAssuranceSanteDTO
- PaiementDTO

4.3. Mappers

Les mappers assurent la conversion entre entités JPA et DTOs. Ils étaient implémentés manuellement à l'aide de BeanUtils.

5. Web Services

REST Les contrôleurs REST exposent les fonctionnalités métier sous forme d'API HTTP. Ils sont annotés avec `@RestController` et `@RequestMapping`. La documentation est générée automatiquement via SpringDoc / OpenAPI (Swagger UI accessible sur `/swagger-ui.html`).

5.1. Endpoints principaux

Méthode	URL	Description
GET	/api/clients	Liste de tous les clients
GET	/api/clients/{id}	Détail d'un client
POST	/api/clients	Créer un client
DELETE	/api/clients/{id}	Supprimer client
GET	/api/contrats	Liste de tous les contrats
GET	/api/contrats/{id}	Détail d'un contrat
POST	/api/contrats/automobile	Créer un contrat automobile
POST	/api/contrats/habitation	Créer un contrat habitation
POST	/api/contrats/sante	Créer un contrat santé

GET	/api/contrats/client/{clientId}	Détail d'un contrat pour un client
PATCH	/api/contrats/{id}/statut	Modifier statut d'un contrat
DELETE	/api/contrats/{id}	Supprimer un contrat
GET	/api/paiements/contrats/{contratId}	Détail d'un paiement
POST	/api/paiements	Créer un paiement
DELETE	/api/paiements/{id}	Supprimer un paiement

6. Sécurité — Spring Security et JWT

La sécurité de l'application repose sur Spring Security combiné à JSON Web Token (JWT). Ce mécanisme est sans état (stateless) : aucune session HTTP n'est maintenue côté serveur.

6.1. Mécanisme d'authentification

- Le client envoie ses identifiants (username / password) sur l'endpoint POST /auth/login.
- Spring Security vérifie les credentials via UserDetailsService.
- En cas de succès, un token JWT est généré et retourné au client.
- Pour chaque requête suivante, le client inclut le token dans le header :
Authorization: Bearer <token>.
- Un filtre JwtFilter intercepte la requête, valide le token et injecte l'utilisateur dans le SecurityContext.

6.2. Rôles et autorisations

Trois rôles sont définis avec des niveaux d'accès distincts :

Rôle	Droits accordés
ROLE_CLIENT	Consulter ses propres contrats et paiements

ROLE_EMPLOYE	Créer et modifier des contrats, enregistrer des paiements
ROLE_ADMIN	Accès complet : gestion des utilisateurs, de tous les contrats

6.3. Composants Spring Security

- SecurityConfig : configuration de la chaîne de filtres, règles d'autorisation par URL et rôle.
- JwtUtil : génération, validation et extraction des claims du token JWT.
- JwtFilter : filtre OncePerRequestFilter qui intercepte chaque requête.
- UserDetailsServiceImpl : chargement des utilisateurs depuis la base de données.